

PyTorch 2.x: A Definitive Framework-Agnostic Technical Integration Reference

Scope & currency: This reference covers PyTorch itself, generically, as of June 2026. The latest stable release is **PyTorch 2.12.0** (released May 13, 2026; 2,926 commits from 457 contributors since 2.11), per PyPI and the [2.12 release blog](#). Minimum Python is 3.10. Canonical sources: pytorch.org/docs, github.com/pytorch/pytorch.

TL;DR

- PyTorch 2.x is a hardware-agnostic tensor + autograd platform whose modern integration surface is `torch.export` → **ExportedProgram** for portability, `torch.compile` (**TorchDynamo** + **TorchInductor**) for performance, and **torchao** for quantization; TorchScript is now fully deprecated (PyTorch 2.9) and TorchServe is in limited maintenance (archived August 2025).
- The framework-agnostic deployment story rests on three pillars: **ExportedProgram** (serialized graph IR), **AOTInductor** (compiled shared library for Python-free C++ serving), and **ExecuTorch** (`.pte` for edge), plus **ONNX** via the new `dynamo=True` exporter — all built on the same `torch.export` capture.
- For production, prefer `torch.compile` for training/JIT inference, AOTInductor for server-side Python-free inference, ExecuTorch for mobile/edge, and external runtimes (vLLM, TensorRT) for LLM serving rather than the deprecated TorchServe.

Key Findings

- Version state (mid-2026):** Stable is 2.12.0; the 2.x series has matured `torch.compile` into a near-default for serious training, deprecated TorchScript, and unified hardware support across CUDA/ROCm/XPU/MPS. PyTorch 2.12 adds a device-agnostic `torch.accelerator.Graph` API and `torch.export.save` support for Microscaling (MX) quantization formats.
- Compilation is the headline 2.x capability:** `torch.compile` (one line) wraps TorchDynamo (bytecode-level graph capture via CPython's PEP 523 frame-eval API), AOT Autograd (backward graph capture), and TorchInductor (Triton for GPU, C++/OpenMP for CPU).
- Export replaces scripting:** `torch.export.export` produces an **ExportedProgram** (ATen-level graph) that is the single source for ONNX, AOTInductor, ExecuTorch, and PT2E quantization.
- Quantization moved to torchao:** PT2E (export-based) quantization and the `quantize_` API in torchao are the modern path; eager and FX graph modes still exist but torchao is where int4/float8/MX work lives.

5. **Distributed has shifted to per-parameter sharding:** FSDP2 (`fully_shard`) uses DTensor-based dim-0 sharding; DDP remains the baseline for replicated data parallelism.

Details

1. Core tensor library

`torch.Tensor` is the central multi-dimensional array, analogous to `numpy.ndarray` but with GPU acceleration and autograd. Reference: [torch.Tensor docs](#).

```
python

import torch
x = torch.randn(3, 4)                # CPU float32 tensor
y = torch.ones(3, 4, dtype=torch.bfloat16)
z = x @ y.float().T                  # matmul
print(z.shape, z.dtype, z.device)
```

Device management. PyTorch supports CPU, CUDA (NVIDIA), ROCm (AMD, exposed through the `cuda` device API), MPS (Apple Silicon), and XPU (Intel). The 2.x idiom is the device-agnostic `torch.accelerator` API. See [torch.cuda](#), [MPS backend](#).

```
python

device = torch.accelerator.current_accelerator() if torch.accelerator.is_available()
x = torch.randn(1024, 1024, device=device)
```

Dtypes. Floating: `float64/32/16`, `bfloat16`; reduced-precision float8 (`torch.float8_e4m3fn`, `torch.float8_e5m2`, plus `e4m3fnuz`, `e5m2fnuz`, `e8m0fnu`); integer `int64/32/16/8`, `uint8`. As of PyTorch 2.3 (`uint1`–`uint7`) and 2.6 (`int1`–`int7`), sub-byte integer dtypes exist but are **placeholder/"shell" dtypes with no working core ops** — real low-bit compute is provided by torchao tensor subclasses plus bitpacking. There is also `torch.float4_e2m1fn_x2` (packs two FP4 values) used in MX/NV formats. Reference: [torchao quantization overview](#). (PyTorch)

Autograd. PyTorch uses reverse-mode automatic differentiation over a dynamically-built directed acyclic graph (DAG) of `Function` objects. Tensors with `requires_grad=True` track history; `.backward()` traverses the graph in reverse topological order accumulating `.grad` on leaves. The graph is rebuilt every forward pass (define-by-run), which is what allows arbitrary Python control flow. References: [autograd mechanics](#), [torch.autograd](#), [autograd tutorial](#).

```
python
```

```
x = torch.tensor([2.0], requires_grad=True)
y = x ** 3
y.backward()
print(x.grad)          # dy/dx = 3x^2 = 12
```

Custom autograd Function

```
class Square(torch.autograd.Function):
    @staticmethod
    def forward(ctx, i):
        ctx.save_for_backward(i)
        return i * i
    @staticmethod
    def backward(ctx, grad_out):
        (i,) = ctx.saved_tensors
        return grad_out * 2 * i
```

Gradient computation can be disabled with `torch.no_grad()` or, more aggressively, `torch.inference_mode()`. Gradients accumulate, so the optimizer's `zero_grad()` is required each step.

2. Model authoring — `torch.nn`

`torch.nn.Module` is the base class. Submodules, `Parameter`s (learnable, registered in `state_dict`), and `buffer`s (non-learnable persistent state, e.g. BatchNorm running stats) are auto-registered. References: [torch.nn](#), [nn.Module](#), [torch.nn.functional](#), [torch.nn.init](#).

python

```
import torch.nn as nn
import torch.nn.functional as F

class MLP(nn.Module):
    def __init__(self, d_in, d_hidden, d_out):
        super().__init__()
        self.fc1 = nn.Linear(d_in, d_hidden)
        self.fc2 = nn.Linear(d_hidden, d_out)
        self.register_buffer("call_count", torch.zeros(1))
        nn.init.kaiming_normal_(self.fc1.weight, nonlinearity="relu")

    def forward(self, x):
        self.call_count += 1
        return self.fc2(F.relu(self.fc1(x)))
```

The functional API (`torch.nn.functional`) provides stateless ops; `nn` modules wrap them with managed parameters.

3. Training

Optimizers (`torch.optim`): SGD, Adam, AdamW, Adagrad, RMSprop, etc. Three implementations exist with performance ordering **fused** > **foreach** > **for-loop**; foreach (multi-tensor) is the default when applicable. PyTorch 2.12 added `fused=True` for Adagrad, joining Adam, AdamW, and SGD. Reference: [torch.optim](#).

python

```
opt = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=0.01, fused=True)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=1000)
```

Canonical training loop with gradient clipping, AMP, gradient accumulation, and LR scheduling:

python

```
from torch.amp import autocast, GradScaler
scaler = GradScaler("cuda")
accum_steps = 4
for step, (x, y) in enumerate(loader):
    x, y = x.to(device), y.to(device)
    with autocast(device_type="cuda", dtype=torch.float16):
        loss = loss_fn(model(x), y) / accum_steps
    scaler.scale(loss).backward()
    if (step + 1) % accum_steps == 0:
        scaler.unscale_(opt)
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        scaler.step(opt)
        scaler.update()
        opt.zero_grad(set_to_none=True)
        sched.step()
```

Mixed precision (`torch.amp`): `autocast` runs eligible ops in lower precision; `GradScaler` prevents fp16 gradient underflow. The device-namespaced `torch.amp.autocast("cuda"|"cpu")` / `torch.amp.GradScaler("cuda")` API supersedes the deprecated `torch.cuda.amp.*` forms. bf16 typically needs no scaler. Backward passes should run outside the autocast context. References: [torch.amp](#), [AMP recipe](#), [AMP examples](#).

Loss functions are in `torch.nn` (e.g. `CrossEntropyLoss`, `MSELoss`) / `torch.nn.functional`.

4. Data — `torch.utils.data`

`Dataset` (map-style `__getitem__`/`__len__` or `IterableDataset`), `DataLoader` (batching, shuffling, multiprocessing workers, pinned memory), `Sampler`/`BatchSampler`, and `collate_fn`.

Reference: [torch.utils.data](#).

```
python

from torch.utils.data import Dataset, DataLoader

class MyDataset(Dataset):
    def __init__(self, X, y): self.X, self.y = X, y
    def __len__(self): return len(self.X)
    def __getitem__(self, i): return self.X[i], self.y[i]

loader = DataLoader(MyDataset(X, y), batch_size=64, shuffle=True,
                    num_workers=4, pin_memory=True, drop_last=True)
```

For reproducibility with multiprocessing loading, set `worker_init_fn` and a `generator`. On Windows/macOS the default start method is `spawn`, so wrap launch code in `if __name__ == "__main__":` and keep `collate_fn`/dataset definitions top-level.

5. Distributed and parallel training

References: [torch.distributed](#), [DDP notes](#), [DDP tutorial](#), [FSDP2 tutorial](#), [DTensor / device mesh](#).

DistributedDataParallel (DDP) — multi-process, one process per GPU; replicates the model and all-reduces gradients in buckets during backward.

```
python

import os
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP

dist.init_process_group(backend="nccl")           # launched via torchrun
local_rank = int(os.environ["LOCAL_RANK"])
torch.cuda.set_device(local_rank)
ddp_model = DDP(model.to(local_rank), device_ids=[local_rank])
```

Launch with `torchrun --nproc_per_node=8 --nnodes=1 train.py`.

FSDP2 (**fully_shard**) — per-parameter dim-0 sharding represented as DTensor (replaces FSDP1's flat-parameter sharding). Apply bottom-up (layers before root). Supports `MixedPrecisionPolicy`, `CPUOffloadPolicy`. FSDP2 does not directly support full state dicts.

```
python
```

```
from torch.distributed.fsdp import fully_shard, MixedPrecisionPolicy
from torch.distributed.device_mesh import init_device_mesh

mesh = init_device_mesh("cuda", (dist.get_world_size(),))
for layer in model.layers:
    fully_shard(layer, mesh=mesh, mp_policy=MixedPrecisionPolicy(param_dtype=torch.bf16))
fully_shard(model, mesh=mesh)
```

Tensor/pipeline parallelism and device meshes are built on DTensor and `DeviceMesh`, enabling N-D parallelism (data × tensor × pipeline). TorchTitan demonstrates these for LLM pretraining. PyTorch 2.12 also previews `torchcomms` as a future default communications layer.

6. Performance and compilation

`torch.compile` is the unified entry point. Reference: [torch.compiler](#), [Dynamo overview](#).

```
python
```

```
compiled = torch.compile(model, mode="max-autotune") # or "reduce-overhead", "default"
out = compiled(x)
```

- **TorchDynamo** hooks CPython's frame evaluation API (PEP 523) to rewrite bytecode and extract FX graphs; unsupported constructs cause "graph breaks" (it falls back to eager for that fragment). `fullgraph=True` forces an error on any break. Guards trigger recompilation when assumptions (shapes, types) change; excessive recompilation hits `torch._dynamo.config.recompile_limit`. (PyTorch + 3)
- **AOT Autograd** traces the backward graph ahead of time and min-cut partitions forward/backward.
- **TorchInductor** lowers FX graphs to **Triton** kernels (GPU) or **C++/OpenMP** (CPU), with operation fusion as a key optimization. ATen is the underlying operator library. (Medium)
- **Mega Cache** (`torch.compiler.save_cache_artifacts`/`load_cache_artifacts`) enables portable end-to-end compile caching across machines. (PyTorch)

`torch.fx` is the Python-to-Python graph capture/transform toolkit underlying much of this: `symbolic_trace` builds a `Graph` of `Node`s wrapped in a `GraphModule`. Reference: [torch.fx](#).

```
python
```

```
import torch.fx as fx
gm = fx.symbolic_trace(model)
for node in gm.graph.nodes:
    if node.op == "call_function" and node.target is torch.add:
        node.target = torch.mul
gm.recompile()
```

TorchScript (`torch.jit`) is deprecated. As of PyTorch 2.9 TorchScript is fully deprecated; `torch.jit.script`/`trace` emit deprecation warnings directing users to `torch.compile` or `torch.export`. It remains for legacy loading but receives no new features. References: [torch.jit](#), [TorchScript deprecation note](#).

7. Quantization

References: [PT2E quantization \(torchao\)](#), [quantization overview](#), [torchao](#).

Three historical modes: **eager mode** (manual), **FX graph mode** (`quantize_fx`, semi-automatic), and **PT2E** (export-based, the modern graph path). Schemes: **dynamic** (weights quantized ahead, activations at runtime), **static** (both quantized, requires calibration), and **QAT** (fake-quant in training). int8 is the workhorse; int4 and below are increasingly common for LLM weights.

PT2E flow (export → prepare → calibrate/train → convert → lower):

```
python

import torch
from torchao.quantization.pt2e.quantize_pt2e import prepare_pt2e, convert_pt2e
from torchao.quantization.pt2e.quantizer.x86_inductor_quantizer import (
    X86InductorQuantizer, get_default_x86_inductor_quantization_config)

m = torch.export.export(model.eval(), example_inputs).module()
quantizer = X86InductorQuantizer().set_global(get_default_x86_inductor_quantization_config())
m = prepare_pt2e(m, quantizer)
m(*example_inputs) # calibration
m = convert_pt2e(m)
optimized = torch.compile(m) # lower via Inductor
```

torchao is the modern home of quantization (current stable ~0.17; 0.18 upcoming). Its `quantize_` API swaps in quantized tensor subclasses and works with `torch.compile` and FSDP2:

```
python
```

```
from torchao.quantization import quantize_, Int4WeightOnlyConfig, Int8WeightOnlyConfig
quantize_(model, Int8WeightOnlyConfig()) # int8 weight-only
quantize_(model, Int4WeightOnlyConfig(group_size=32)) # int4 tinygemm path
```

Config objects (e.g. `Int4WeightOnlyConfig`, `Int8DynamicActivationInt4WeightConfig` for ExecuTorch, `Float8DynamicActivationFloat8WeightConfig` for Hopper+) have superseded the older function-style `int4_weight_only`/`int8_weight_only`. Reported inference gains, per the torchao README, include "Int4 weight-only: 1.73x speedup with 65% less memory for Gemma3-12b-it on H100 with slight impact on accuracy" and "Float8 dynamic quantization: 1.5-1.6x speedup on gemma-3-27b-it and 1.54x and 1.27x speedup on Flux.1-Dev and CogVideoX-5b respectively on H100 with preserved quality." [arXiv](#)

Low-bit / ternary networks. Sub-byte integer dtypes are shell dtypes; real low-bit compute uses torchao tensor subclasses + bitpacking. **BitNet b1.58 (ternary, weights in {-1,0,+1}, ≈1.58 bits/param)** is supported only as a **prototype/community** effort within torchao (bitpacked into `uint2`), code-generated with `torch.compile`), not a stable workflow. torchao provides generic bitpacking kernels and 1-8 bit ARM CPU kernels. [GitHub](#) [PyPI](#)

Microscaling (MX) formats. MXFP4/MXFP6/MXFP8 use a block (size 32) sharing an E8M0 scale (`torch.float8_e8m0fnu`) with `float4_e2m1fn_x2`/float8 data, implemented in `torchao.prototype.mx_formats` (prototype). **PyTorch 2.12's** contribution is narrow but important: `torch.export.save`/`load` now serialize the `float8_e8m0fnu` block-scale dtype, "unblocking the full export-to-deployment workflow for models leveraging Microscaling quantization" (2.12 blog). [PyTorch](#)

Float8 training. `torchao.float8.convert_to_float8_training(model)` recursively converts `nn.Linear` to `Float8Linear`; with `torch.compile` and FSDP2 the torchao float8 README reports "e2e pretraining speedups of up to 1.5x at 512 GPU / 405B parameter count scale, and up to 1.25x at 8 GPU / 8B parameter count scale, with performance and accuracy validated on up to 2k GPUs, via torchtitan's float8 integration." Three recipes (`tensorwise`, `rowwise`, `rowwise_with_gw_hp`) trade speed vs accuracy. [PyTorch](#) [GitHub](#)

8. Export and interoperability

`torch.export.export` captures a full-graph `ExportedProgram` (ATen IR, no graph breaks allowed) — the foundation for deployment and the recommended replacement for TorchScript. It supports dynamic shapes via `Dim`. References: `torch.export`, `ExportedProgram`.

python


```
ep = torch.export.export(model, (example_input,),
                          dynamic_shapes=({0: torch.export.Dim("batch")},))
torch.export.save(ep, "model.pt2")
loaded = torch.export.load("model.pt2").module()
```

ONNX. As of PyTorch 2.5+, `torch.onnx.export(..., dynamo=True)` is the recommended exporter (built on `torch.export`/FX); the legacy TorchScript-based exporter is deprecated. In PyTorch 2.9 the dynamo path became the **default** for `torch.onnx.export`. The dynamo exporter uses FakeTensorMode for dramatically lower export-time memory. References: [torch.onnx, export-based ONNX exporter](#).

```
python

onnx_program = torch.onnx.export(model, (example_input,), dynamo=True)
onnx_program.save("model.onnx")
```

ExportedProgram and ONNX are the two standard interchange formats for framework-agnostic deployment.

9. Serving and deployment

AOTInductor — compiles an ExportedProgram to a self-contained shared library (`.so`)/(`.pt2`) for Python-free C++ inference; maintains a stable libtorch C ABI for backward compatibility across versions. Best for server-side inference where CPython overhead or the GIL is unacceptable. No CUDA graphs support. References: [AOTInductor](#), [AOTInductor Python runtime recipe](#). [GitHub](#)

```
python

ep = torch.export.export(model, example_inputs)
path = torch._inductor.aoti_compile_and_package(ep)  # -> .pt2 artifact
m = torch._inductor.aoti_load_package(path)
```

ExecuTorch — PyTorch's edge/on-device runtime (GA'd as ExecuTorch 1.0). Export → compile/quantize/partition to backends → `.pte` → tiny C++ runtime; per the [ExecuTorch runtime docs](#), "the core runtime library is less than 50kB when built without kernels or backends," and the [executorch repo](#) advertises "12+ Hardware Backends — Open-source acceleration for Apple, Qualcomm, ARM, MediaTek, Vulkan, and more." It powers on-device AI at Meta.

```
python
```

```
from executorch.exir import to_edge_transform_and_lower
from executorch.backends.xnnpack.partition.xnnpack_partitioner import XnnpackPartitioner
ep = torch.export.export(model.eval(), example_inputs)
prog = to_edge_transform_and_lower(ep, partitioner=[XnnpackPartitioner()]).to_executorch()
open("model.pt", "wb").write(prog.buffer)
```

libtorch / C++ frontend — pure C++17 API (`torch::nn`, `torch::optim`, autograd) mirroring Python, plus `torch::jit::load` for serialized modules. For low-latency, multithreaded, or embedded use. References: [C++ frontend](#), [cppdocs](#).

TorchServe is in limited maintenance — the [pytorch/serve](#) repository was archived August 7, 2025; per the project: "no planned updates, bug fixes, new features, or security patches. Users should be aware that vulnerabilities may not be addressed." For new deployments prefer AOTInductor/ExecuTorch for PyTorch-native paths, or external serving (vLLM, NVIDIA Triton/TensorRT-LLM, LitServe) — vLLM in particular speaks PyTorch natively and dominates LLM serving. [GitHub + 2](#)

10. Ecosystem and integration surfaces

PyTorch integrates with external systems via **standard interfaces** rather than bespoke couplings: ExportedProgram and ONNX for model interchange; REST/gRPC fronting an inference runtime; model registries storing `.pt2`/`.pte`/ONNX artifacts. The **Hugging Face** stack (transformers, accelerate, PEFT, TRL, diffusers) is PyTorch-first and the dominant model source; HF `transformers` provides its own FX tracer and integrates torchao quantization. This keeps the reference framework-agnostic: any consumer that can load an ExportedProgram, an ONNX graph, an AOTInductor `.pt2`, or an ExecuTorch `.pte` can serve PyTorch models without the authoring framework. [Tech Insider](#)

11. Reproducibility

References: [reproducibility notes](#), [torch.use_deterministic_algorithms](#).

```
python

import torch, random, numpy as np
torch.manual_seed(0); random.seed(0); np.random.seed(0)
torch.use_deterministic_algorithms(True)
torch.backends.cudnn.benchmark = False # deterministic cuDNN algo selection
# For CUDA >= 10.2: set env CUBLAS_WORKSPACE_CONFIG=:4096:8
```

Determinism is not guaranteed across devices, CUDA versions, or PyTorch releases, and deterministic algorithms are typically slower. `fill_uninitialized_memory` defaults to True under deterministic mode. For DataLoader, seed workers via `worker_init_fn`. AOT compilation

(export/AOTInductor) adds discipline by freezing the graph: exported models must have no recompiles, so data-dependent control flow and optional/variant inputs must be handled explicitly.

Recommendations

Staged adoption:

1. **Author** standard eager `nn.Module` code; add `torch.compile(model)` early — if it runs clean with `fullgraph=True`, you have a graph-capturable model that will export cleanly later. Benchmark eager vs compiled; expect meaningful GPU speedups on Ampere/Hopper/Blackwell. *Threshold to escalate:* if you hit frequent graph breaks or recompilations, profile with `TORCH_LOGS=recompiles` and refactor data-dependent control flow.
2. **Train** with `torch.amp` (bf16 preferred on Ampere+; fp16 + GradScaler otherwise), `AdamW(fused=True)`, gradient clipping, and a scheduler. Scale out with DDP first; move to **FSDP2** when the model or optimizer state exceeds single-GPU memory. *Threshold:* if a single replica OOMs, switch DDP→FSDP2; if communication dominates, tune sharding granularity and enable mixed-precision/CPU-offload policies.
3. **Quantize** with **PT2E + torchao** once accuracy targets are set; use int8 static/dynamic for general models, int4/float8 via torchao `quantize_` for LLM weights. Validate accuracy on a held-out set before/after. *Threshold:* if int8 accuracy drop is unacceptable, move to QAT (`prepare_qat_pt2e`).
4. **Deploy** along the export-centric path: `torch.export` → AOTInductor (server, Python-free C++), ExecuTorch (mobile/edge), or ONNX (`dynamo=True`) for cross-runtime portability. Do **not** start new projects on TorchScript or TorchServe. For LLM serving, use vLLM/TensorRT-LLM. *Threshold:* if export fails on control flow, use `torch.cond`/explicit dynamic shapes, or fall back to a `torch.compile`-served Python process.

Version discipline: pin the PyTorch version and matching domain libraries

(torchvision/torchaudio) per the [compatibility matrix](#); AOTInductor artifacts rely on the stable C ABI but should be regenerated and revalidated on major upgrades. Note PyTorch 2.6 changed `torch.load` to default `weights_only=True` (security hardening). [GitHub](#)

Caveats

- **Forward-looking items:** PyTorch 2.12's `torchcomms`-by-default and Inductor support for `torch.cond` in CUDA graphs are stated as **planned for 2.13+** — treat as roadmap, not shipped. MX-format compute in torchao remains **prototype**; the 2.12 release adds only export *serialization* of the MX scale dtype, not new MX kernels. (As a concrete data point, the PyTorch blog "Accelerating 2K scale pre-training up to 1.28x with TorchAO, MXFP8 and TorchTitan on Crusoe B200 Cluster" reports for Llama3-70B (HSDP2, CP=2): "Our tests

showed successful loss curve equivalence and speedups between 1.22x and 1.28x as compared to training in BF16, even at the full 1856-GPU scale" — note this used an earlier torchao version.)

- **Deprecations to avoid in new code:** TorchScript (`torch.jit`), fully deprecated 2.9), `torch.cuda.amp.*` (use `torch.amp`), the TorchScript-based ONNX exporter (`dynamo=False`), TorchServe (archived/limited maintenance). TorchText is discontinued.
- **Sub-byte dtype nuance:** `torch.int4`/`uint4` and friends are **placeholder dtypes** in core with no working ops — do not assume native int4 compute; it lives in torchao subclasses. BitNet/ternary support is community/prototype, not a supported PyTorch workflow.
- **Reproducibility limits:** identical seeds do **not** guarantee identical results across CPU/GPU, hardware, CUDA versions, or releases.
- **Source quality:** version numbers and dates here are anchored to official PyPI/GitHub/pytorch.org sources; some ecosystem figures (e.g. third-party "research share" percentages, vendor speedup claims) are secondary and were treated as such. torchao's exact current PyPI version (0.17 vs 0.18) should be confirmed at install time.